

Pruebas de Rendimiento con Locust
Asignatura: Ingeniería de Software II

Presentado por: Oswin Olsman Cuaran
Edwin Hernan Chenas Escobar
Docente: Albeiro Montes

Universidad Nacional de Colombia

2026

Introducción.

El presente informe documenta el desarrollo de un taller de pruebas de rendimiento con Locust, aplicado a una API REST desarrollada en Laravel 9. Durante el proceso se realizaron ajustes en la API y se verificó el funcionamiento de sus principales endpoints mediante solicitudes HTTP.

Posteriormente, se busca evaluar el comportamiento del servicio mediante pruebas de carga, estrés y capacidad, considerando los tiempos de respuesta, la cantidad de peticiones y los errores presentados. A partir de los resultados obtenidos, se identificarán posibles limitaciones y oportunidades de mejora en el funcionamiento de la API.

4. Modificaciones en la API

Antes de realizar las pruebas de rendimiento, se hicieron algunos ajustes en la API con el propósito de mejorar la consulta de información y evitar que el sistema tuviera que procesar todos los registros al mismo tiempo.

Se implementó la paginación para consultar los usuarios por grupos de hasta 100 registros, se trasladó el filtro de edad a la base de datos y se estableció un orden para mostrar los resultados. También se ajustó la información que devuelve cada consulta y se habilitó la lectura de los parámetros necesarios para navegar entre páginas.

Estos cambios buscan reducir la cantidad de información procesada en cada solicitud y facilitar la evaluación del rendimiento de la API mediante Locust.

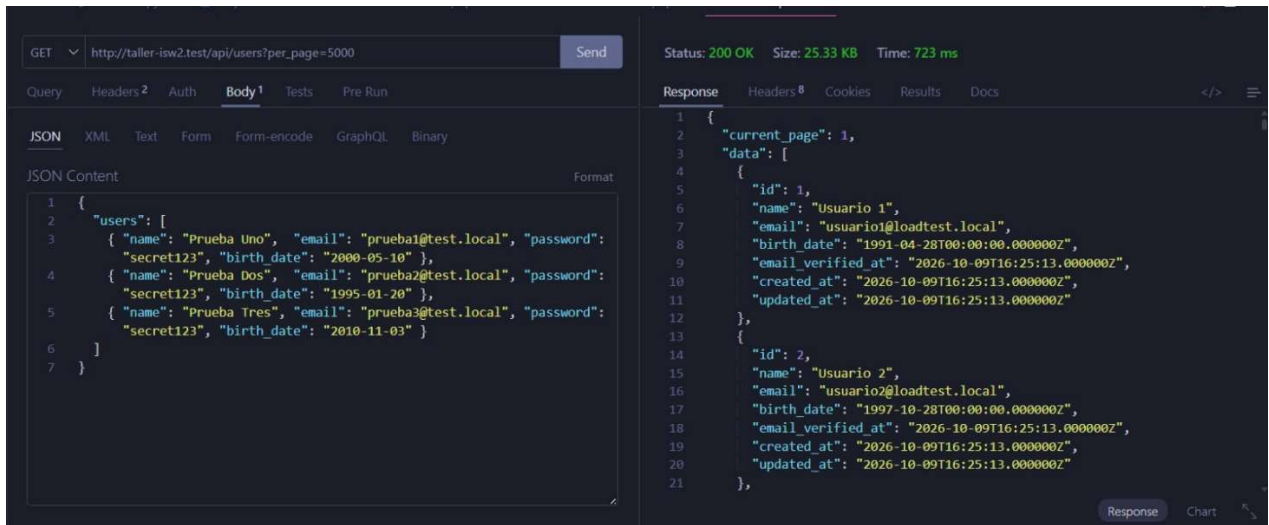
4.1 Ajuste de límite de Solicitudes.

Se modificó la configuración del límite de solicitudes de Laravel para permitir la ejecución de las pruebas de rendimiento con Locust. Inicialmente, la API permitía 60 peticiones por minuto por dirección IP, lo que ocasionaba respuestas de error 429 debido a que los usuarios simulados compartían la misma IP.

Para evitar que esta restricción interfiriera con las pruebas, se desactivó temporalmente el límite de solicitudes. De esta manera, se busca evaluar la capacidad de respuesta de la API sin que esta protección limite artificialmente las peticiones generadas durante las pruebas.

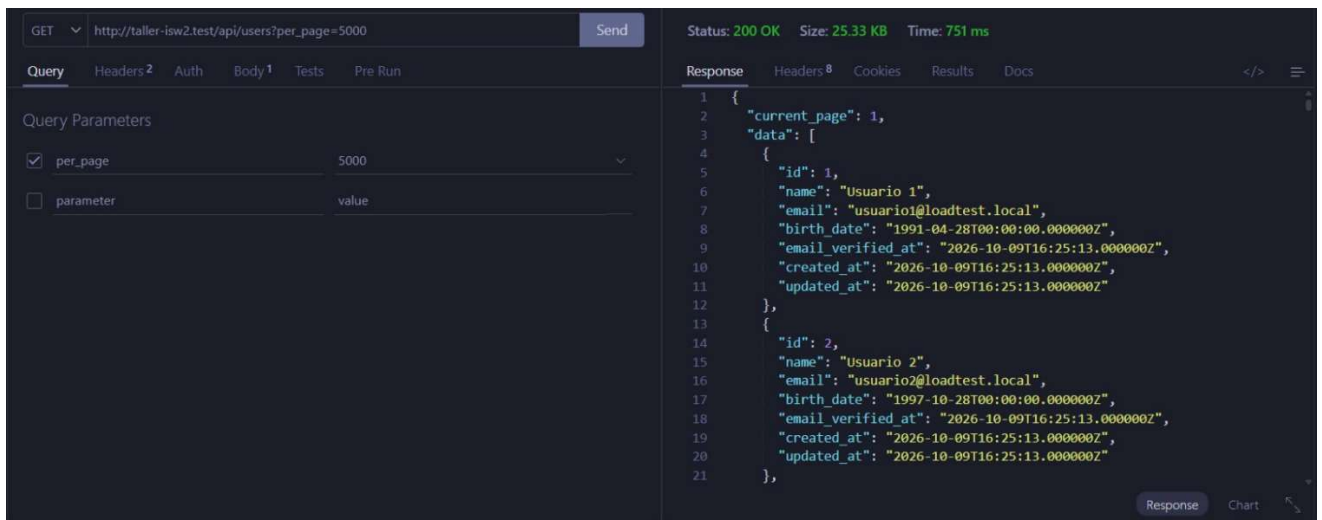
4.2 Configuración del límite de registros.

Se verificó el comportamiento de la API al solicitar 5.000 registros mediante el parámetro `per_page=5000`. La solicitud obtuvo una respuesta **200 OK**, con un tiempo de 723 ms. Sin embargo, la captura muestra que la respuesta continúa en la primera página, por lo que se debe comprobar en el código o en la respuesta completa si el límite máximo de 100 registros se está aplicando correctamente.



Evidencia del limite máximo de registros.

Se realizó una nueva solicitud al endpoint GET /api/users utilizando el parámetro per_page=5000. La API respondió con el código 200 OK, un tiempo de respuesta de 751 ms y un tamaño de respuesta de 25,33 KB. Estos resultados son consistentes con la aplicación de un límite en la cantidad de registros devueltos, aunque para confirmarlo se debe verificar el campo per_page en la respuesta JSON.

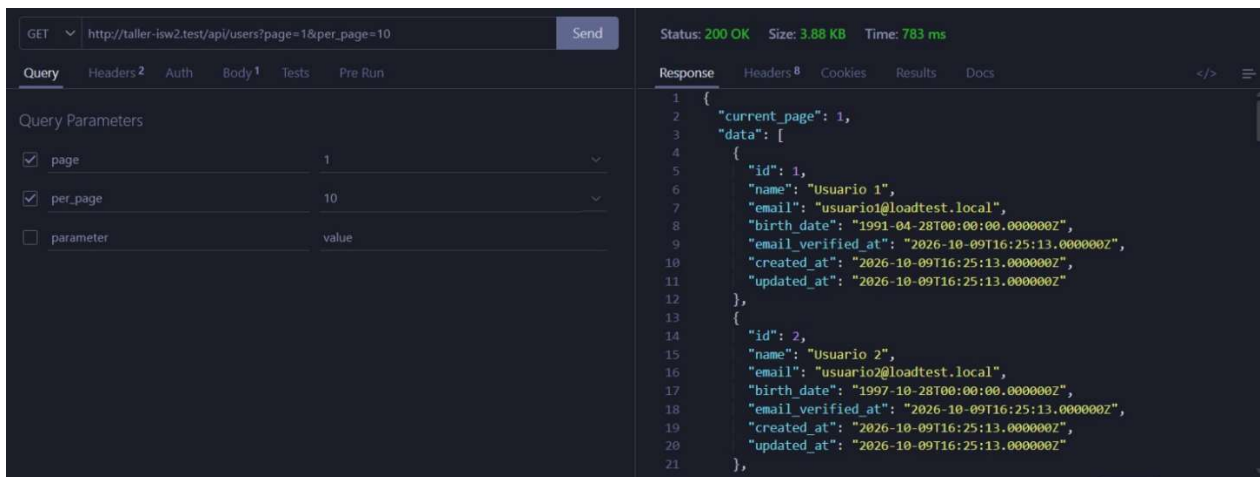


5. Verificación de Endpoints

5.1 Consulta paginada de usuarios.

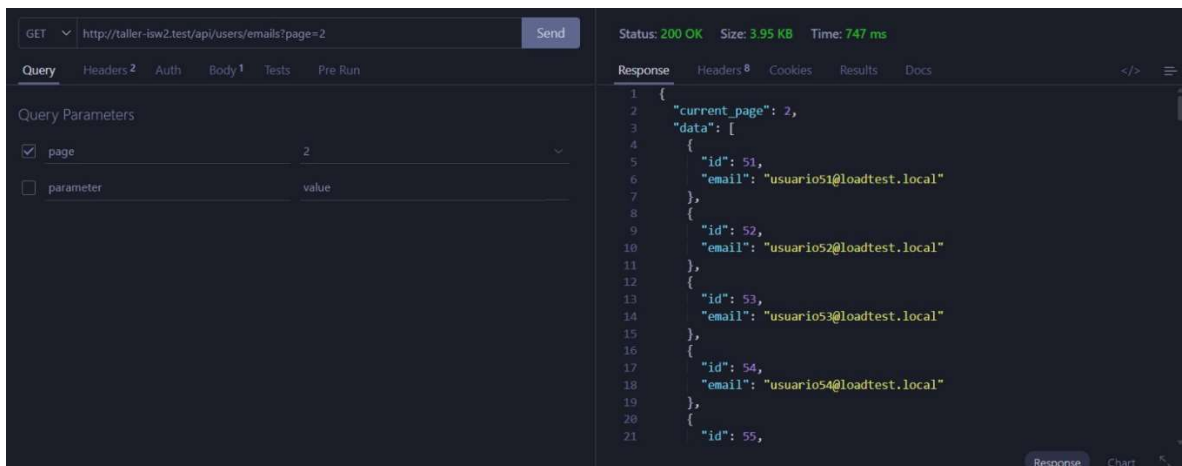
Se realizó una prueba del endpoint GET /api/users mediante Postman, utilizando los parámetros page=1 y per_page=10 para consultar la primera página con 10 usuarios.

La solicitud obtuvo una respuesta **200 OK**, con un tiempo de respuesta de 810 ms. En el resultado se observa la información de los usuarios junto con el campo data, lo que permite verificar el funcionamiento de la consulta paginada.



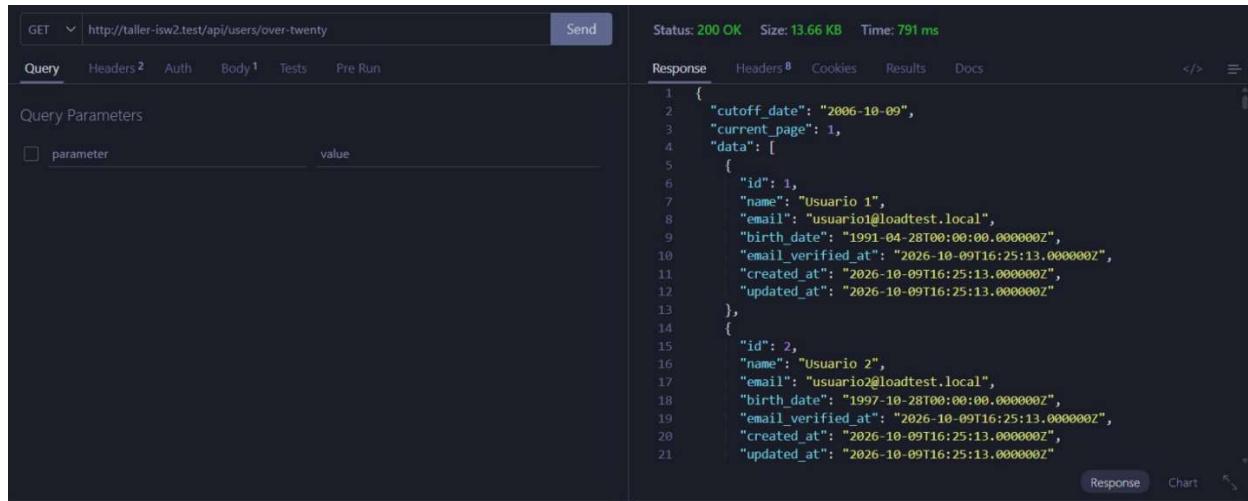
5.2 Consulta paginada de correos electrónicos.

Se realizó una prueba del endpoint GET /api/users/emails mediante Postman, utilizando el parámetro page=2 para consultar la segunda página de resultados. La solicitud obtuvo una respuesta 200 OK, con un tiempo de respuesta de 798 ms. La respuesta muestra los identificadores y correos electrónicos de los usuarios, lo que permite verificar el funcionamiento de este endpoint.



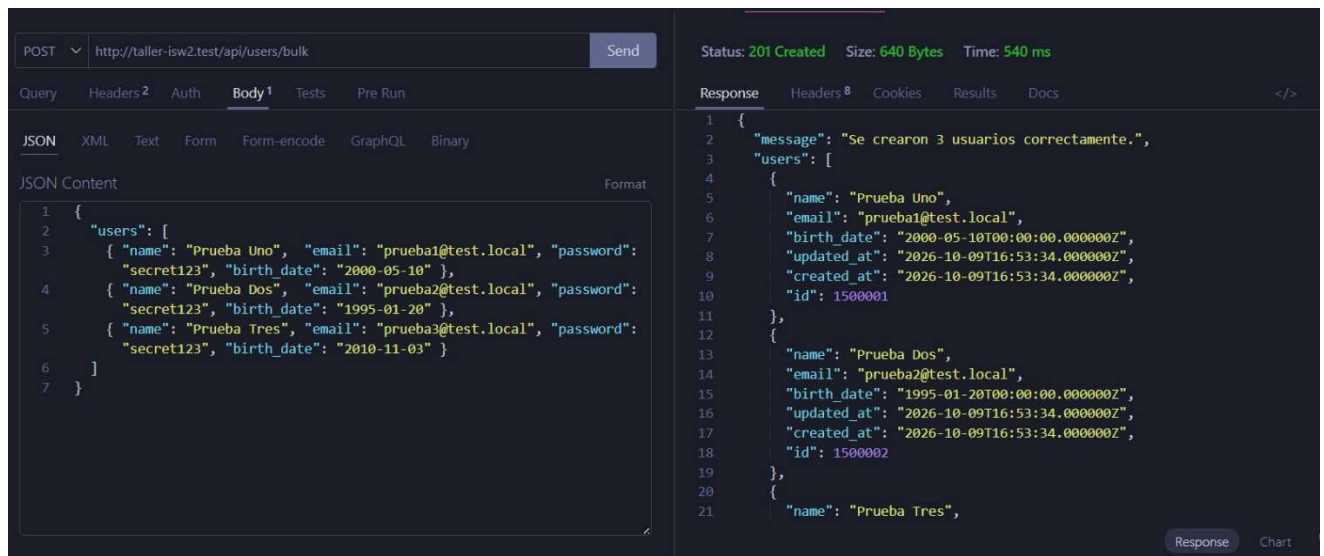
5.3 Consulta de Usuarios mayores de 20 años.

Se realizó una prueba del endpoint GET /api/users/over-twenty mediante Postman para consultar los usuarios que cumplen con el criterio de edad establecido. La solicitud obtuvo una respuesta **200 OK**, con un tiempo de respuesta de 766 ms. En la respuesta se observa el campo cutoff_date y la información de los usuarios consultados, lo que permite verificar el funcionamiento del endpoint.



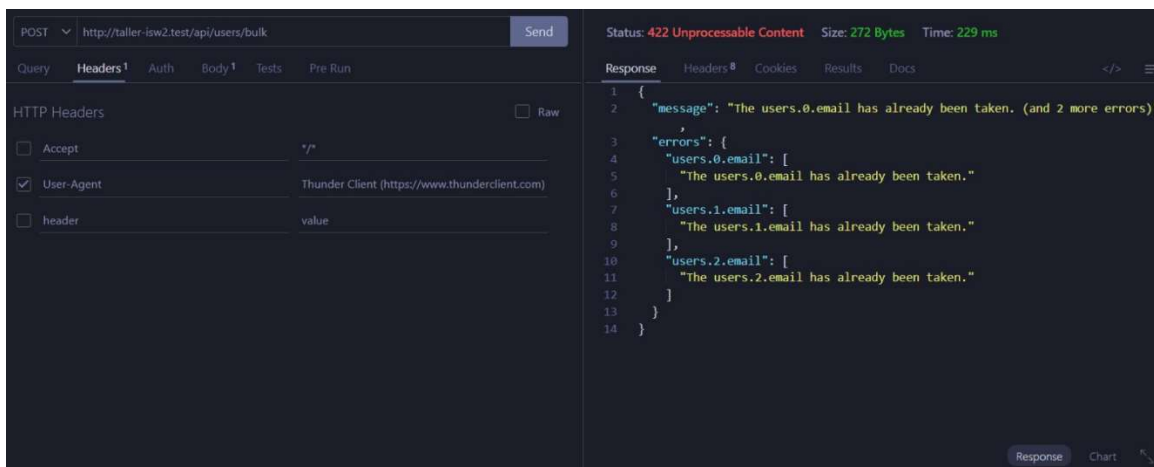
5.4 Creación simultanea de usuarios.

Se realizó una prueba del endpoint POST /api/users/bulk mediante Postman, enviando en una sola solicitud los datos de tres usuarios. La API respondió con el código **201 Created** y confirmó que los tres registros fueron creados correctamente. El tiempo de respuesta fue de 540 ms, lo que permite verificar el funcionamiento de esta operación de inserción múltiple.



Validación de correos electrónicos duplicados.

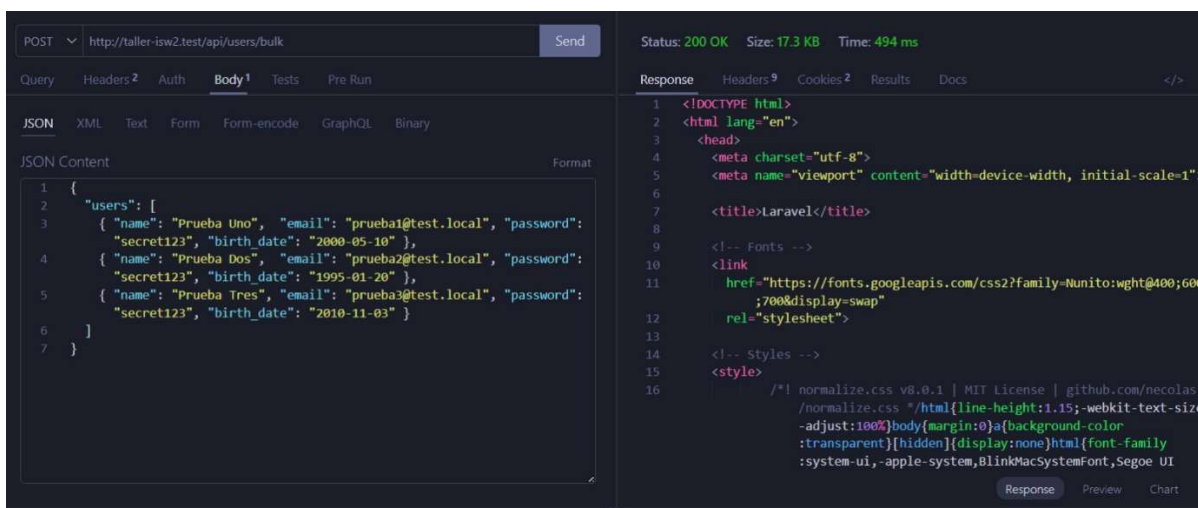
Se realizó una nueva prueba del endpoint POST /api/users/bulk utilizando correos electrónicos que ya estaban registrados en la base de datos. En esta ocasión, la API respondió con el código 422 Content e indicó cuáles correos estaban duplicados. Esto permite comprobar que la validación funciona correctamente y evita registrar usuarios con direcciones de correo existentes.



Prueba de validación de correos duplicados.

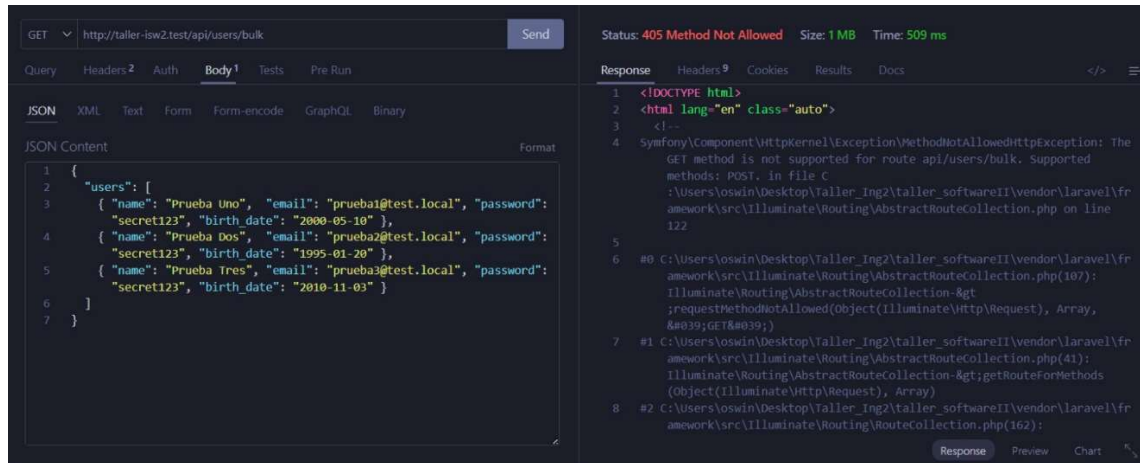
Se realizó una prueba enviando usuarios con correos electrónicos que ya existían en la base de datos. La validación no permitió completar la operación; sin embargo, como la solicitud no indicó que esperaba una respuesta JSON, Laravel redirigió a una página web. Por este motivo, Thunder Client mostró una respuesta 200 OK con contenido HTML, en lugar del código 422 esperado para este tipo de error.

Este resultado evidencia la importancia de configurar correctamente las solicitudes para recibir respuestas JSON y verificar que los errores de validación se informen de forma adecuada.



Prueba de método HTTP no permitido.

Se realizó una solicitud GET al endpoint /api/users/bulk, aunque esta operación está configurada para aceptar únicamente solicitudes POST. La API respondió con el código 405 Method Not Allowed, indicando que el método utilizado no está permitido para esta ruta. Esta validación ayuda a evitar el uso incorrecto del endpoint de creación de usuarios.



5.5 Resumen de las verificaciones realizadas.

Las pruebas realizadas con Postman permitieron comprobar el funcionamiento de los cuatro endpoints y verificar el comportamiento de la paginación. Las solicitudes obtuvieron respuestas satisfactorias, con tiempos entre 540 ms y 810 ms.

Prueba	Resultado	Tiempo
Consulta paginada de usuarios	200 OK	810 ms
Consulta de correos electrónicos	200 OK	798 ms
Consulta de usuarios mayores de 20 años	200 OK	766 ms
Creación de tres usuarios	201 Created	540 ms
Solicitud con per_page=5000	200 OK	723 ms

En la última prueba, la respuesta tuvo un tamaño de 25,33 KB. Según la revisión compartida por el equipo, esto corresponde a un máximo de 100 usuarios por página, aunque se solicitaban 5.000. Para dejar constancia de esta validación, se

debe incluir una captura donde se observe el campo `per_page`: 100 en la respuesta JSON.

También se observó que las consultas GET presentaron tiempos similares, aunque el tamaño de sus respuestas fuera diferente. El equipo relaciona este comportamiento con el conteo total de registros que realiza la paginación sobre la base de datos. Esta explicación deberá tenerse en cuenta al analizar el rendimiento de la API.

6. Pruebas de rendimiento de Locust.

La captura muestra una ejecución de Locust 2.46.7 contra la API `http://taller-isw2.test`, con una rampa de incorporación de usuarios que pasó por 200, 500 y finalmente 5.000 usuarios, a una tasa de 3 usuarios por segundo en la última etapa.

6.1 Resultados de ejecución.

En la tabla de Locust se registraron los siguientes resultados:

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
GET	GET /api/users	477	92	5600	7100	7900	4898.61	794	8596	11509.55	7.8	4.3
GET	GET /api/users/emails	370	63	5800	7100	7600	5004.21	793	8233	3579.12	6.6	3.6
GET	GET /api/users/over-twenty	252	50	2200	4500	5500	2559.42	726	5719	11644.43	4.8	3.3
POST	POST /api/users/bulk	119	29	1800	2700	3600	1731.54	465	4662	574.03	2.4	1.6
Aggregated		1218	234	4800	6900	7600	4137.29	465	8596	8059.96	21.6	12.8

ABOUT

```
(venv) PS C:\Users\oswin\Desktop\Taller_Ing2\taller_softwareII\locust> locust -f locustfile.py --host=http://taller-isw2.test
[2026-10-09 12:37:18,376] Oswin/INFO/locust.main: Starting locust 2.46.7
[2026-10-09 12:37:18,376] Oswin/INFO/locust.main: Starting web interface at http://localhost:8089, press enter to open your default browser.
[2026-10-09 12:40:09,350] Oswin/INFO/locust.runners: Ramping to 200 users at a rate of 1.00 per second
[2026-10-09 12:42:25,337] Oswin/INFO/locust.runners: Ramping to 500 users at a rate of 3.00 per second
[2026-10-09 12:43:11,648] Oswin/INFO/locust.runners: Ramping to 5000 users at a rate of 3.00 per second
```

Durante la ejecución de Locust se observó que los tiempos de respuesta aumentaron frente a las pruebas individuales realizadas con Thunder Client. De las 1.218 solicitudes registradas, 234 presentaron fallos, equivalentes aproximadamente al 19,2 % del total.

El endpoint de consulta de correos electrónicos presentó el mayor tiempo promedio de respuesta, con aproximadamente 5 segundos, seguido del listado general de usuarios. Estos resultados muestran que las consultas GET fueron las operaciones con mayor tiempo de respuesta promedio en esta ejecución.

Sin embargo, para determinar las causas de los errores y evaluar la capacidad real de la API, es necesario revisar los mensajes de fallo y confirmar si la prueba alcanzó los 5.000 usuarios simulados.

6.1 Prueba de estrés.

En la prueba de estrés se incrementó progresivamente la cantidad de usuarios simulados para observar el comportamiento de la API ante una mayor demanda. En las gráficas de Locust se evidencia un aumento en los tiempos de respuesta, con valores que alcanzan aproximadamente los 7.000 ms en el percentil 95.

También se observan variaciones en las mediciones durante la ejecución, lo que indica que el sistema presentó cambios en su comportamiento a medida que aumentaba la carga. Estos resultados permiten analizar la respuesta de la API y deben complementarse con los datos de errores y solicitudes registrados al finalizar la prueba.



